University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Concession Test, 2021

Answer all questions.
Approximate marks per question are shown in brackets.

Time: 45 minutes **Marks: 45**

Section 1: ASD [20]

Question 1. **[2]**

Consider version control systems. Why is repository an appropriate architecture for their implementation?

Version control stores large volumes of change logs for software in repositories[1] which needs to be accessed by one or many members assigned to the version control repository[1]

Question 2. **[3]**

Explicitly designing and documenting software architecture at the outset of a project may take a long time. Explain the purpose of this documentation and name two advantages of doing this.

Documenting software architecture provides a high-level picture of the system structure, which people from different disciplines, who are involved in the system development process, can readily understand[1]

- Award one mark for listing any of the advantages identified below.

1. **Stakeholder communication** The architecture is a high-level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
2. **System analysis** Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
3. **Large-scale reuse** An architectural model is a compact, manageable description of how a system is organized and how the components interoperate. The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse. As I explain in Chapter 15, product-line

architectures are an approach to reuse where the same architecture is reused across a range of related systems.

Question 3. **[3]**

Consider the concept of major and minor releases in software. Explain how the two releases are different from one another, and what types of changes (and bug fixes) might go in each. Where do software architectures factor into the differences?

A minor release is a small piece of functionality which would not normally require a change management process and should have minimal bug fixes. Minor releases are further unlikely to affect the systems architecture. Major releases adds a large set of features which would normally be agreed upon during a change management process and might demand changes to the current system architecture.

Question 4. **[3]**

Imagine that you are a peripheral developer for an open-source project. What are the steps by which you will use git to contribute code to a project?

Create a branch[1], commit changes to your branch and submit this branch for code review by another developer[1], who will merge the changes if it is acceptable[1].

Question 5. **[6]**

A common risk in software engineering projects is missed deadlines due to poor estimates or unanticipated delays. Define the 3Ms of risk management and use them to explain how you might address this risk. State any assumptions.

Mitigation: How to avoid or reduce the risk, or assume the risk and accept and control it. [2]

Monitoring: Track factors to enable us to determine if the risk is becoming more or less likely. [2]

Management: Make contingency plans to deal with the risk if it becomes a reality. [2]

Question 6. **[2]**

What is the purpose of refactoring software, and when is it typically done?

Refactoring should be done to optimize code and to make it more maintainable [1] and should be done throughout the lifespan of software[1]

Question 7. **[1]**

Give an example of a Behavioral design pattern and explain its purpose.

Observer [0.5]

Behavior Design Patterns are responsible for how one class communicates with other.[0.5]

Section 2: Theory of Computation [15]

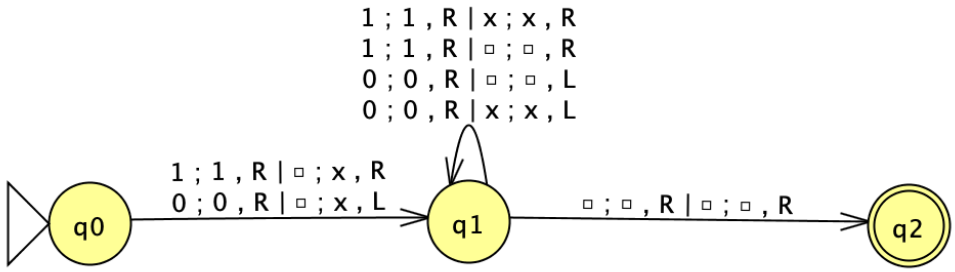
Question 8. **[6]**

Design a Turing Machine that accepts an unequal number of 0s and 1s. Thus, it should accept strings like 00111 and 10110, but reject strings like 0011 and 1010. You may choose to do that with any type of Turing Machine that has been covered in the module.
Provide the *full definition* of the TM. The transition function may be drawn (using consistent common notation), listed in a list of functions (adhering to the definition of the function), or shown in a table with the states (first column the states, first row the alphabet).

ANSWER

This is fairly similar to exercise 8.2.2 and 8.4.6, but then the other way around (*un*even rather than even). The multi-tape is the easiest, and instead of making it go to the final state upon the # (or whatever the marker is on the scratch tape), it should go to the final state if it the cell with # is *not* under the tape head, i.e., if the cell on the scratch tape has a blank.

Here's a drawing in JFLAP, where the box is the blank (b) and the x is the marker on the scratch tape:



Note for marking: either the diagram or the table, or as a list of functions (delta(q0, [0,B]) = (q1, [0, x], [R,L]) etc etc). For the definition: M = ({q0,q1,q2},{0,1},{0,1,B},q0,q2)

Marks: 1 for listing all the components, then for the remaining 5: give 5 for everything correct, 2.5 if it does about half of it, 0 if wrong (accepts a different language, incomplete). Deduct 0.5 mark for inconsistent notation, for missing the arrow/forgetting to indicate the start symbol, forgetting arrows in the transitions.

END ANSWER.

Question 9. **[4]**

L1 reduces to L2. We know that L2 is a recursive language. What can be said about L1? Prove it.

ANSWER

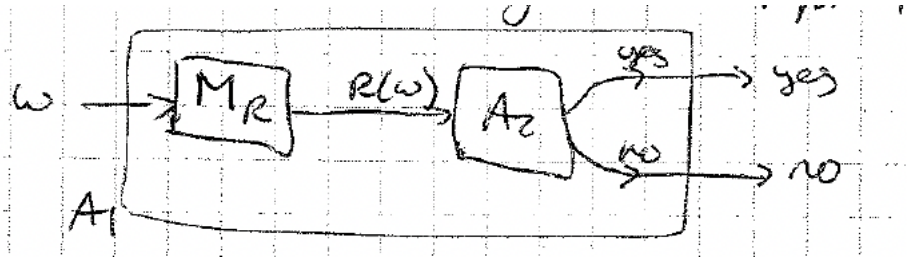
L1 is also recursive. [1 mark]

Proof]:

L2 is recursive, so has an algorithm for it, which returns y/n for sure [1 mark].

Since it does reduce (given in statement), there must be an algorithm A_R for that reduction converting w in to ‘R(w)’ (A2’s input) [1 mark].

Then we can construct an algorithm A1 such that M1’s input is converted to one for M2 for A2, and we can use A2’s output for A1’a output. So then w in L1 equiv R(w) in L2 [1 mark]. Schematically, optionally.alternatively:



qed

NOTE FOR MARKING: just the figure plus a note that w in L1 equiv R(w) in L2 would be fine as well to gain the 3 marks for the proof. missing the “w in L1 equiv R(w) in L2” would lose 0.5 marks.

END ANSWER

Section 3: Theory of Algorithms [10]

Question 10. **[6]**

Consider the following algorithm:

```
ALGORITHM what(int [] A, int i, int j)
   if i == j then
      return A[i]
   if i == j-1 then
      if A[i] < A[j] then
         return A[i]
      else
         return A[j]

   k = (i+j) / 2
   m1 = what(A, i, k)
   m2 = what(A, k+1, j)
   if m1 < m2 then
      return m1
   else
      return m2
```

i) What does the algorithm above do? What strategy (algorithmic pattern) does this algorithm use? [2]

ii) Determine the performance of this algorithm by setting up a recurrence relation for the basic operations being performed and solving it using the Master Theorem (see Appendix A). Show all your work. [4]

i) It uses recursion to find the minimum value in an array [1]. It employs a divide and conquer strategy [1].

ii) Basic operation is comparison [1]. In the worst case 3 comparisons are done on each step and 2 recursive calls, each of which operates on half the array, thus:

$C(n) = 2\ C(n/2) + 3$ [1]

In terms of the Master Theorem a = 2, b = 2, d = 0 [1].

This fits the $a > b^d$ case so efficiency is

$\Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$ [1].

Question 11. **[4]**

Given the following adjacency matrix:

$$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

i) Show the steps in the execution of Warshall's algorithm for transitive closure applied to this matrix. [3]

ii) From the outcome, list a pair of nodes (a, b) where node b is not reachable from node a. Note that a and b should be numbers in the range 1, …, 3. [1]

Solution:

i)

$R1 = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ (no change)

[1]

$R2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

[1]

$R3 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

[1]

ii) either (2, 1) or (3, 1). [1]

Appendix A – Master Theorem

If we have a recurrence of this form: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$